

A New Approach to Improved PNB-based Attacks on Reduced-round ChaCha and Salsa

Goutam Paul, Anup Kumar Kundu, and Sucheta Chakrabarti

Indian Statistical Institute, Kolkata,
203 Barrackpore Trunk Road, Kolkata 700108, INDIA.
goutam.paul@isical.ac.in, anupkundumath@gmail.com,
suchetadrdo@hotmail.com

Abstract. ChaCha and Salsa are two ARX based stream ciphers which are widely used in data encryption including TLS v1.3 standard, VPN software etc. Exploiting Probabilistic Neutral Bits (PNB) is one of the most significant cryptanalysis strategies for reduced round versions of these ciphers. The seminal work using PNB by Aumasson et al. (FSE 2008) claims that the PNB set mostly depends on the output bit difference occurring in the intermediate round. The subsequent works mainly relied on the differential or differential-linear cryptanalysis, or multiple distinct input-output differentials for which the bias is higher than a threshold in the intermediate round. In this paper, we propose a new PNB set construction based on multiple output bit differences with respect to a single input bit difference only. We exploit the differentials to mount key recovery attacks using a multi-step procedure depending on our new PNB set. Our attack achieves a time complexity of $2^{167.9008}$ for ChaCha20/7 and $2^{183.5361}$ for Salsa20/8, beating all the existing PNB-based attacks on ChaCha20/7 and Salsa20/8 by a significant margin. Further, both our time and data complexities for ChaCha20/7.5 are better than the latest published work by Flórez-Gutiérrez and Todo (Eurocrypt 2025). We have also verified our attack experimentally on a published toy version of ChaCha (FSE 2023).

Keywords: Stream Cipher, ChaCha, Salsa, ARX Cipher, Probabilistic Neutral Bit (PNB), Bias, Differential attack, Differential-Linear attack.

1 Introduction

Symmetric key cryptography is used in a wide range of hardware and software products, such as data encryption, secure communication, disk encryption, database security, VPN software etc. In the symmetric key paradigm, stream ciphers are faster than block ciphers, and therefore are more widely used in practical applications and hardware implementations.

Salsa [5] is a stream cipher, designed by Bernstein in 2005 and was a candidate in the eStream project [1]. The original version of the cipher has 20 rounds. There are some other variant of the cipher also consists of 8 (Salsa20/8) and 12

rounds (Salsa20/12). The version Salsa 20/12 was accepted in the phase 3 of the project by receiving the highest votes in 2008. Since the cipher was proposed as a candidate in the eStream project, many attacks [2],[10],[19],[22],[27],[30],[31] have been mounted on the cipher.

ChaCha [6] is another stream cipher which was also proposed by Bernstein in the year 2008. The structure of the cipher is similar to Salsa and the only difference lies in the round function. The `quarterround` operation of the round function of ChaCha applies on the rows and the diagonals of the state matrix instead of the rows and the columns, as are in Salsa. Due to this, the diffusion in the state bits is also higher in ChaCha than in Salsa.

1.1 Summary of Existing Works

Most of the previously reported cryptanalysis of the above ciphers start by giving an input difference in the initial state of the cipher and then finding the corresponding output difference in the j -th bit at the end of the r -th intermediate round. The next step is to perform a meet-in-the-middle approach by first reaching the reduced final round R of the targeted cipher, and then coming back to the r -th intermediate round, $r < R$, by applying the inverse function $(R - r)$ -th times. The first attack using this strategy was proposed by Crowley [10] to find a truncated differential for 5 rounds of Salsa by going 3 forward rounds, and then coming back for 2 rounds. Later, Aumasson et al. [2] introduced the idea of Probabilistic Neutral Bit (PNB) and proposed an attack for 8 rounds of Salsa by going 4 forward rounds and coming back for 4 rounds. For ChaCha, the attack [2] worked for 7 rounds with 3 forward and 4 backward rounds respectively.

Afterwards, many authors explored different variants of PNB set formation and improved the attack complexity [24],[23],[14],[32],[29]. Some other works have used predefined size of PNB [18],[3] as well. The work [30] extended the previously proposed PNB work by column and row chaining distinguishers. They further proposed new second order differential trails and generalized the notion of probabilistic neutral bits (PNB) to probabilistic neutral vectors (PNV) by flipping more than one input bit simultaneously and improved the attack complexity.

Some PNB-based works [7],[9],[15],[4],[8],[13],[34],[3],[17] explored the combination of the differential and linear cryptanalysis, some of which have penetrated into fractional rounds of the ciphers. Also, the last two works used an additional technique of the hamming weight ($\mathcal{HW}$) restriction of the state matrix.

In [4], the authors found a new differential-linear distinguisher by giving 2-bit differences at the beginning of the differential part of the differential-linear trail. They further used Mixed Integer Linear Programming (MILP) tool to find trails for the linear part. Using this, they found 5 round differential-linear trail and provided a key recovery attack for 7 rounds.

In [7], the authors proposed the first multi-bit (different input-output difference at each time) differential-linear strategy to find the forward bias after 3.5, 4, 4.5 rounds of ChaCha and 5 rounds of Salsa cipher. In addition to multiple input-output differentials, the work [12],[21] used a systematic multi-step

approach to maximize the size of the PNB set based on neutrality measurement parameter.

In [20], the authors have introduced a novel theoretical methodology, called bit puncturing, as an alternative to the PNB technique against the cipher ChaCha. This bit puncturing approach is based on theoretical analysis of the Walsh spectrum, which aims to construct an approximated key-recovery map by ignoring certain input bits. The purpose is to reduce the number of active input bits from the secret key, to simplify the complexity of the key recovery procedure. This reduces the time complexity to 2^{40} times than the previous best work for 7 rounds of ChaCha. However, they did not give any verification of their attack approach using any toy version of the cipher.

The attack complexities of relevant important existing works are presented in Table 1 in comparison with our results. Note that, the work proposed in [20] used a different approach than PNB. Hence we do not compare the above work with our proposed method.

1.2 Our Contributions

Here we enlist our main contributions one by one.

1. In general, all the existing PNB based differential / differential linear cryptanalysis on ChaCha and Salsa utilize only a specific output bit-difference for any single input bit-difference. More specifically, all the existing works either explored single input-output ($ID-OD$) differential or multiple input-output (ID_i-OD_i) differentials with linear combinations. We first investigate the differential distinguisher in the unexplored domain of single input-multiple output ($ID-OD_i$) differentials.
2. In the seminal work on the PNB based attacks, the authors mention the following [2, Section 3.5].

“The number n of PNB 's for some fixed threshold γ mostly depends on the output difference, but not on the input difference.”

We construct a new PNB set (PNBS) by taking the union of proper choice of PNB sets, each of which is related to separate output bit-differences corresponding to a single input bit-difference. Our construction method is described in Section 3 and Algorithm 2. Thus, the choice of the input bit-difference also play a significant role in determining the PNB set, contrary to the claim of the seminal work [2] and all subsequent follow-ups.

3. Further, we partition this new type of PNB set into suitably crafted disjoint subsets to mount a divide-and-conquer algorithm on those parts, leading to an improved key recovery attack on reduced-round ChaCha and Salsa. This partitioning method is detailed in Section 4 and Algorithm 3. In particular, we show that with proper choice of input bit-difference and attack parameters, our method achieves key recovery of 7 round, 7.5 round of ChaCha with a time complexity of $2^{167.9008}$, $2^{237.4147}$ and that of 8 round Salsa with a time complexity of $2^{183.5361}$. The comparison of our results for ChaCha7 and Salsa8 with respect to all the previous works, is given in Table 1.

Cipher	Rounds	Attack Strategy	Data	Time	References	Year
ChaCha	7	Differential	2^{27}	2^{248}	[2]	2008
			$2^{23.89}$	$2^{238.94}$	[23]	2016
			$2^{103.3}$	$2^{210.3}$	[32]	2023
			$2^{68.9008}$	$2^{167.9008}$	This work	2025
		Differential-Linear	2^{28}	2^{233}	[7]	2016
			$2^{32.87}$	$2^{235.2}$	[18]	2017
			2^{224}	2^{224}	[9]	2021
			$2^{48.83}$	$2^{230.86}$	[3]	2022
			$2^{90.20}$	$2^{221.95}$	[15]	
			$2^{87.18}$	$2^{218.92}$	[16]	2023
			$2^{110.81}$	$2^{206.8}$	[4]	
			$2^{93.79}$	$2^{192.89}$	[13]	2024
			$2^{102.63}$	$2^{189.7}$	[34]	
			$2^{101.09}$	$2^{178.12}$	[29]	
Salsa	8	Differential	2^{31}	2^{251}	[2]	2008
			$2^{32.11}$	$2^{247.2}$	[24]	2015
			$2^{22.46}$	$2^{245.52}$	[23]	2016
			$2^{61.5361}$	$2^{183.5361}$	This work	2025
		Differential-Linear	2^{96}	$2^{244.9}$	[7]	2016
			$2^{30.86}$	$2^{243.7}$	[18]	2017
			$2^{113.14}$	$2^{217.14}$	[8]	2023
			$2^{117.29}$	$2^{210.32}$	[12]	2024
			$2^{99.73}$	$2^{186.01}$	[17]	2025

Table 1. Comparison of data and time complexities of our PNB based key recovery attack with the existing ones of 7 and 8 rounds of ChaCha and Salsa respectively.

4. An obvious way to compute the backward bias for $\mathcal{ID} - \mathcal{OD}_i$ is to put random values in the corresponding PNB set P_i and actual values to the rest. However, as validated from our experiments with ToyChaCha, such assignments do not guarantee the independence of forward and backward biases which is a crucial requirement of the attack model. Rather, our observation establishes that we need to put random bits in the union of all PNBs to com-

pute the backward bias for any $\mathcal{ID} - \mathcal{OD}_i$'s (irrespective of the particular P_i). This is a non-trivial generalization to handle multiple $\mathcal{OD}$ s from a single $\mathcal{ID}$.

5. We also verify the independence assumption and our key recovery attack procedure on a toy version of ChaCha, namely, ToyChaCha [14].
6. We have also used both differential and differential-linear strategies, to apply our attack on ChaCha7.5. We have observed that, by using only differential approach (without any additional linear or intermediate condition) we can attack ChaCha7.5. This has not been done in any of the previous works to the best of our knowledge. The approaches are discussed in Section 5.2. For both the approaches, the data and the time complexities become $2^{53.4147}$ and $2^{237.4147}$ using our attack procedure. Both our data and time complexities of ChaCha7.5 outperform the latest reported work proposed in [20].

Advantage of Our Work. The advantage of our new construction procedure is derived from the following observations.

- The cardinality of our new PNB set is larger than that of the single input-output bit-difference case.
- It creates more partitions (≥ 2) of the key space.
- The key recovery procedure is used the partitions in step by step manner. This reduces the overall time complexity.

The advantage of choosing multiple output differences from the same input difference is that, in this case we can reuse the same generated keystream pairs to recover the key bits in the key recovery attack. Hence the key recovery attack can be performed more efficiently in a multi-step procedure, which is discussed later.

Organization of the paper. The notations used in the whole paper along with the description of the ciphers Salsa and ChaCha are described in Section 1. In Section 2.4, we revisit the original PNB work by Aumasson et al. and discuss the terminologies and definitions used to find PNB set and mount the key recovery attack. Section 3 discusses how we construct our new PNB set to mount an improved attack. Our key recovery attack procedure is presented in Section 4. The final attack parameters and complexity results for all the ciphers, are given in Section 5. The verification on a toy version of ChaCha is presented in Section 6. Finally, we conclude the paper in Section 7.

2 Preliminaries and Background

In this section, first we introduce the notations used throughout the paper, followed by brief descriptions of ChaCha and Salsa. Then we revisit the existing notions, terminologies and PNB construction strategies.

2.1 Notations

The notations used in this paper are defined as follows. The initial state matrix of the cipher is denoted by X , while $X^{(r)}$ represents the state matrix after the r -th round of the cipher. The element $X_{i,j}^{(r)}$ corresponds to the j -th bit of the i -th word in the state matrix at round r . All operations are considered over $\mathbb{F}_2^{32}$, where $x + y$ denotes the addition of x with y while $x - y$ denotes the subtraction of y from x . The circular left rotation of x by n bits is represented by $x \lll n$. The keyspace of the cipher is represented by $\mathbb{K}$. For the key recovery attack, Th_u denotes the threshold used in key recovery, N_u and T_u indicate the data and time complexities respectively. The number of randomly chosen initialization vectors (IVs) is specified by $NumIV$, and $NumKey$ denotes the number of randomly chosen keys in all our experiments.

Consider the two states $X_{p,q}^{(r)}$ and $X'_{p,q}{}^{(r)}$ at some q -th bit of p -th word at the r -th round. We denote difference of the states by $\Delta_{p,q}^{(r)} = X_{p,q}^{(r)} \oplus X'_{p,q}{}^{(r)}$. Now, for an input difference ($\mathcal{ID}$), we denote it by $\Delta_{p,q}^{(0)}$ and mean, a bit difference is given in the q -th bit of the p -th word at initial round. The output difference ($\mathcal{OD}$) at some r -th round is denoted by $\Delta_{p,q}^{(r)}$. This means, a bit difference appears with a certain bias at $X_{p,q}^{(r)}$ in the r -th intermediate round.

2.2 Salsa

Salsa [5] is a stream cipher that consists of R rounds with $R = 20$. It has state size 512 bits. A Salsa state can be represented by a 4×4 matrix, called a state matrix. Each cell of the state matrix consists of 32-bits, called a word. The initial state matrix of the cipher has 4 words of predefined round constants $c = (c_0, c_1, c_2, c_3)$, 8 words of key $k = (k_0, \dots, k_7)$, and 4 words of initial vector (IV) $v = (v_0, v_1, v_2, v_3)$. The initial state matrix can be represented by the following:

$$X = \begin{pmatrix} x_0 & x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} \\ x_{12} & x_{13} & x_{14} & x_{15} \end{pmatrix} = \begin{pmatrix} c_0 & k_0 & k_1 & k_2 \\ k_3 & c_1 & v_0 & v_1 \\ v_2 & v_3 & c_2 & k_4 \\ k_5 & k_6 & k_7 & c_3 \end{pmatrix}.$$

Depending upon the key sizes 128-bit and 256-bit, the cipher has two variants. In the 256-bit version, distinct 256-bit key is given in 8 words of the initial state. In the other one, 128-bit key is given repeatedly in (k_0, k_1, k_2, k_3) and (k_4, k_5, k_6, k_7) . The round function of the cipher has **columnround** / **rowround** function depending upon the even / odd round. Each of these functions calls another function, named **quarterround**, which is applied on the 4 words of the columns for **columnround** and rows for **rowround** in the corresponding state. This process is repeated for R rounds to generate the final state. Each **quarterround** function updates 4 words of 32-bit as the following:

$$b = b \oplus ((a + d) \lll 7), \quad (1)$$

$$c = c \oplus ((b + a) \lll 9), \quad (2)$$

$$d = d \oplus ((c + b) \lll 13), \quad (3)$$

$$a = a \oplus ((d + c) \lll 18). \quad (4)$$

Finally after adding the initial state with the final one i.e. $Z = X + X^{(20)}$, the keystream bits are generated.

Inverse function. The decryption function of the cipher differs in the **quarterround**. The inverse of the **quarterround** function updates 4 words (a, b, c, d) by applying the Eqs. (1), (2), (3), (4) in reverse order. After subtracting the initial state from the generated keystream within modulo 2^{32} , the inverse **rowround** and inverse **columnround** functions are called. Like the encryption procedure, inverse **columnround** (**rowround**) applies inverse **quarterround** in each column (row) in the state of the cipher. After repeating the process for R rounds, the initial state is generated.

2.3 ChaCha

ChaCha [6] is a variant of Salsa, that contains R rounds with $R = 20$. The structure of the cipher is similar to its predecessor **Salsa**. Here it takes round constant (c), key (k), and IV (v) cells and forms the initial state matrix as the following:

$$X = \begin{pmatrix} x_0 & x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} \\ x_{12} & x_{13} & x_{14} & x_{15} \end{pmatrix} = \begin{pmatrix} c_0 & c_1 & c_2 & c_3 \\ k_0 & k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 & k_7 \\ v_0 & v_1 & v_2 & v_3 \end{pmatrix}.$$

It has also two versions **ChaCha128** and **ChaCha256** depending upon 128-bit and 256-bit keys. The difference between these two is similar with the difference between the two versions in **Salsa**. Irrespective of applying the **rowround** and **columnround**, here the **columnround** and **diagonalround** function is called in the even and odd round respectively, for R rounds to generate the final state. A **quarterround** function is applied on each column in **columnround** and diagonal i.e. $(x_0, x_5, x_{10}, x_{15})$, $(x_1, x_6, x_{11}, x_{12})$, (x_2, x_7, x_8, x_{13}) , and (x_3, x_4, x_9, x_{14}) in **diagonalround** of the state respectively. The non-linear **quarterround** function updates the words $a, b, c, d \in \mathbb{F}_2^{32}$, as given in the following:

$$a = a + b, \quad d = d \oplus a, \quad d = d \lll 16, \quad (5)$$

$$c = c + d, \quad b = b \oplus c, \quad b = b \lll 12, \quad (6)$$

$$a = a + b, \quad d = d \oplus a, \quad d = d \lll 8, \quad (7)$$

$$c = c + d, \quad b = b \oplus c, \quad b = b \lll 7. \quad (8)$$

The keystream bits are generated by adding the initial state with the final one. In [3], [25], [7] authors discuss about $(R + 0.25)$ and $(R + 0.5)$ rounds for some R number of rounds. Here they have taken only Eqn. (5) and Eqs. (5), (6) respectively in the `quarterround` function.

Inverse function. Similar to `Salsa`, here also at the time of decryption, firstly the initial state matrix is subtracted from the final keystream bits. Then inverse `columnround` and inverse `diagonalround` is applied periodically on the state matrix depending on the round number for generating the initial state. These functions mainly consist of inverse `quarterround` function that takes 4 words (a, b, c, d) and apply the following operations:

$$b = b \lll 25, \quad b = b \oplus c, \quad c = c - d, \quad (9)$$

$$d = d \lll 24, \quad d = d \oplus a, \quad a = a - b, \quad (10)$$

$$b = b \lll 20, \quad b = b \oplus c, \quad c = c - d, \quad (11)$$

$$d = d \lll 16, \quad d = d \oplus a, \quad a = a - b. \quad (12)$$

Note that, for decryption of $(R + 0.25)$ and $(R + 0.5)$ rounds, eqn. (12) and eqs. (12), (11) are used in the inverse `quarterround` function. Through out the whole paper, we consider the 256-bit key variant for both of these ciphers. However, applying the same techniques in 128-bit key variants is also interesting and can be considered as a future work.

2.4 Revisiting Existing Constructions of Probabilistic Neutral Bits (PNB)

In this section, we first revisit the original Probabilistic Neutral Bits (PNB) based attack on reduced round `ChaCha` and `Salsa` by Aumasson et al. [2].

The PNB set divides the key bits of the cipher into two halves. One contains the neutral n -bits for a particular number of rounds and the other half contains m -significant bits ($m = \text{number of key bits} - n$) of the cipher, so here $m = 256 - n$. Note that, the neutral key bits are probabilistic in nature. Hence the attacker needs to run the experiments for certain number of times to find out the forward bias at the particular intermediate round (r) of the cipher as well as the PNB set. The authors have taken the sample of 2^{24} different nonce and counters for each of the 2^{10} keys in all the experiments carried out [2].

The attack is a meet-in-the-middle type of approach where the attacker initially finds the bits with respect to the median forward bias (ϵ_d^*) in the intermediate round. Then the ϵ_d^* is used to calculate the PNB set from the reduced last round (R) of the cipher.

Forward Biases (ϵ_d). Consider X and X' are two valid input states with an initial input difference ($\mathcal{ID}$) at the j -th bit of the i -th word i.e. $\Delta_{i,j}^{(0)} = 1$. The forward bias (ϵ_d) for a single key after r rounds can be calculated as follows:

$$Pr(\Delta_{p,q}^{(r)} = 1 | \Delta_{i,j}^{(0)} = 1) = \frac{1}{2}(1 + \epsilon_d), \quad (13)$$

where $\Delta_{p,q}^{(r)}$ is the output difference ($\mathcal{OD}$) after r -th round at q -th bit of the p -th word. By doing the same experiments for different keys, a set of ϵ_d 's can be calculated. Now one can compute ϵ_d^* which is the median forward bias of the set of ϵ_d 's. ϵ_d^* is used to find the PNB set and in the key recovery procedure.

PNB and the related biases (γ_k, γ) . Suppose Z and Z' are the corresponding output keystream bits of the cipher after R rounds i.e. $Z = X + X^{(R)}$ and $Z' = X' + X'^{(R)}$. Now to check whether the k -th bit is PNB or not, at first we invert the k -th bit in both X and X' to make the initial states $\bar{X}$ and $\bar{X}'$. Then decrypt $Z - \bar{X}$ and $Z' - \bar{X}'$ for $R - r$ rounds. Suppose Y, Y' are the intermediate states after $R - r$ rounds. Now one can compute the neutrality measure of the k -th key bit (γ_k) by considering the following equation:

$$Pr(\Delta_{p,q}^{(r)} = Y \oplus Y' | \Delta_{i,j}^{(0)} = 1) = \frac{1}{2}(1 + \gamma_k). \quad (14)$$

The encrypted function should run for several random IV s to calculate the bias γ_k for the corresponding key bit k . Now, if $\gamma_k \geq \gamma$, where γ is a predefined high bias, then the k -th bit is included in the PNB set for the related bias γ . In [23] the authors have used 2^{26} random samples of nonce, counters to calculate the neutrality measurement for each key bit and finally generated the PNB set.

Reverse biases (ϵ_a) After recovering the PNB set of the cipher, one has to find the backward bias (ϵ_a) and the median of the backward bias (ϵ_a^*) related to the PNB set for the key recovery attack. Suppose Z and Z' are the keystream bits for the states X and X' respectively with $X' = X + \Delta_{i,j}^{(0)}$. Consider that after setting the $\mathcal{ID}$ at $X_{i,j}^{(0)}$, the $\mathcal{OD}$ appears at $X_{i,j}^{(r)}$ in the r -th intermediate round. Now, generate two states $\hat{X}$ and $\hat{X}'$ by putting random values in the n -PNB bits and the original values in the m -significant key bit positions. Apply the inverse function on the states $Z - \hat{X}$ and $Z' - \hat{X}'$ for $R - r$ rounds from the R -th round. Suppose this yields the states $\hat{Y}$ and $\hat{Y}'$ and $\Gamma_{p,q}$ is the difference of $\hat{Y}, \hat{Y}'$ at $X_{p,q}^{(r)}$. Now calculate the following probability:

$$Pr(\Delta_{p,q}^{(r)} = \Gamma_{p,q} | \Delta_{i,j}^{(0)} = 1) = \frac{1}{2}(1 + \epsilon_a). \quad (15)$$

Here the backward bias ϵ_a is calculated by varying the IV s randomly over $\mathbb{F}_2^{128}$ for a key. The median backward bias ϵ_a^* is calculated from the set of ϵ_a 's, generated by varying the different keys from $\mathbb{K}$.

Distinguisher Event for Key Recovery. For the key recovery attack, the event $\Gamma_{p,q} = 1$ is considered condition to the event $\Delta_{i,j}^{(0)} = 1$. If ϵ is the bias of this event, then one can write:

$$Pr[\Gamma_{p,q} = 1 | \Delta_{i,j}^{(0)} = 1] = \frac{1}{2}(1 + \epsilon). \quad (16)$$

Using the standard probability rule, we have $\Pr(\Gamma_{p,q} = 1 | \Delta_{i,j}^{(0)} = 1) = \Pr(\Gamma_{p,q} = \Delta_{p,q}^{(r)} \wedge \Delta_{p,q}^{(r)} = 1 | \Delta_{i,j}^{(0)} = 1) + \Pr(\Gamma_{p,q} \neq \Delta_{p,q}^{(r)} \wedge \Delta_{p,q}^{(r)} = 0 | \Delta_{i,j}^{(0)} = 1)$. Under the assumption that given $\Delta_{i,j}^{(0)} = 1$, the events $\Delta_{p,q}^{(r)} = 1$ and $\Gamma_{p,q} = \Delta_{p,q}^{(r)}$ are statistically independent, we have $\frac{1}{2}(1+\epsilon) = \frac{1}{2}(1+\epsilon_d) \cdot (1+\epsilon_a) + \frac{1}{2}(1-\epsilon_d) \cdot (1-\epsilon_a)$, giving:

$$\epsilon = \epsilon_d \cdot \epsilon_a. \quad (17)$$

Therefore, if the independence assumptions hold in practice, then ϵ can be estimated as $\epsilon_d \cdot \epsilon_a$, as also noted in [2]. However, if the independence assumptions do not hold, then one has to empirically calculate ϵ by directly observing the distinguisher event $\Gamma_{p,q} = 1$ condition to $\Delta_{i,j}^{(0)} = 1$, over many iterations.

Data and Time Complexities of the attack (N, T). Let $P_{fa} = 2^{-\alpha}$ be the probability of false alarm (Type I error) and $P_{nd} = 0.0013$ be the probability of non-detection (Type II error). The sample size of the corresponding attack can be calculated by using the following formula [2]:

$$N \approx \left(\frac{\sqrt{\alpha \log 4} + 3\sqrt{1 - \epsilon^2}}{\epsilon} \right)^2. \quad (18)$$

As mentioned in [2], ϵ_d , ϵ_a and ϵ may have wide variability because of infeasibility of processing sufficient number of samples and so for the estimate to be robust, the medians ϵ_a^* , ϵ_d^* and ϵ^* of the above variables may be calculated by randomly varying the keys over the keyspace.

Let m denote the number of significant (non-PNB) key bits. The time complexity of the attack is given by [2]:

$$T = 2^m (N + 2^{256-m} \cdot P_{fa}) = 2^m \cdot N + 2^{256-\alpha}. \quad (19)$$

A correction of the above formula is suggested in [15] as follows:

$$T = 2^m \cdot N + 2^{256-\alpha} + 2^{256-m}. \quad (20)$$

Threshold for Key Recovery. Note that the distinguisher event $\Gamma_{p,q} = 1$ condition to $\Delta_{i,j}^{(0)} = 1$ has the probability $p = \frac{1}{2}(1 + \epsilon)$. Hence, for the correct key guess, the number of samples out of N that would satisfy the distinguisher event (we can say that this is the number of *votes* for the guessed key) would have a mean $\mu = Np$ and a standard deviation $\sigma = \sqrt{Np(1-p)}$. One may consider a suitable threshold Th (as a function of μ and σ) for the votes of the candidate keys, such that if the number of votes of a key guess is greater than Th , then it is included in the set of possible keys. According to the standard *3-sigma* rule of estimation, if one takes $Th = \mu - 3\sigma$, then theoretically more than 99% of the time the correct key should be included in this filtered set. By simple substitution, one can observe that:

$$Th = 0.5N(1 + \epsilon) - 1.5\sqrt{N(1 - \epsilon^2)}, \quad (21)$$

where ϵ may be replaced by $\epsilon_d \cdot \epsilon_a$, provided the independence assumption holds.

3 Our Construction of new PNB Set (PNBS)

Here in our proposed method, we use the union of the PNB sets which are generated from the suitable choice of the multiple $\mathcal{OD}$'s in the intermediate round corresponding to an $\mathcal{ID}$ in the initial round. The construction procedure is done in the following two steps.

3.1 Finding Output Differential Set

Suppose the input difference is given as $\Delta_{i,j}^{(0)}$, i.e., the j -th bit of the i -th word in the initial round. After a chosen intermediate round r , we will calculate all output bit positions $X_{p,q}^{(r)}$ where the difference $\Delta_{p,q}^{(r)}$ appears with a high forward bias ϵ_d with respect to a predefined threshold. For this, we randomly choose a key k and an initial vector IV . Now, depending upon key and IV , we construct the initial state matrix X of the cipher. We also construct another state matrix X' after giving the $\mathcal{ID}$ at $X_{i,j}^{(0)}$. Now we run both the states independently for r rounds and calculate the output difference $\Delta_{p,q}^{(r)}$ at any arbitrary but fixed position $X_{p,q}^{(r)}$, with $0 \leq p \leq 15$, $0 \leq q \leq 31$ of the state. If $\Delta_{p,q}^{(r)} = 1$, then there is a difference in that position in the intermediate r -th round. For each randomly chosen key, by varying IV for $NumIV$ times, we calculate the probability of forward bias event (ϵ_d) by Eqn. 13. Repeat this process for $NumKey$ times different keys and generate a set of ϵ_d values. Then after taking the median of these values, we compute ϵ_d^* for each output bit $X_{p,q}^{(r)}$ in the intermediate round.

For random states, ϵ_d^* should be approximately 0. If it is above a certain chosen threshold ϵ'_d , then we call $\Delta_{p,q}^{(r)}$ as $\mathcal{OD}_l$ for some index number l , corresponding to the $\mathcal{ID}$ at $\Delta_{i,j}^{(0)}$. We include $\Delta_{p,q}^{(r)}$, where the high bias appears in an output differential set $\mathcal{B}$. To compute all the biases for ChaCha and Salsa in our experiment, we have considered $NumKey = 2^{10}$ and $NumIV = 2^{35}$ samples. The pseudocode of the process is given in Algorithm 1.

3.2 Finding the PNBS

In the previous works, a key bit is declared as PNB corresponding to a fixed $\mathcal{OD}$ at the r -th intermediate round, if the value of γ_k is greater than a certain predefined threshold γ . Here in our approach, we check whether or not the value of γ_k is greater than the threshold corresponding to any $\mathcal{OD}_t \in \mathcal{B}$. If it occurs, then we include the corresponding key bit k in the new PNB set, PNBS. The construction procedure of the new PNB set using our method is given in Algorithm 2. The schematic diagrams for the previously proposed and our new construction procedure are given in Figure 1 and Figure 2 respectively.

Suppose for a fixed $\mathcal{ID}$ $\Delta_{i,j}^{(0)}$ there are v many $\mathcal{OD}$ positions, namely $\mathcal{OD}_1, \dots, \mathcal{OD}_v$. According to Algorithm 2, if we take $\mathcal{B} = \{\mathcal{OD}_l : 1 \leq l \leq v\}$, the PNBS is the set P . Here P is the new PNB set corresponding to $\mathcal{ID} - \cup_{l=1}^v \mathcal{OD}_l$. It is easy to observe that, $P = \cup_{l=1}^v P_l$ where P_l is the PNB set corresponding to

Algorithm 1 FINDING_OUTPUT_DIFFERENTIAL_SET

Input: ϵ'_d : Threshold for forward bias; $\Delta_{i,j}^{(0)}$: $\mathcal{ID}$ at initial round
Output: $\mathcal{B}$, a output differential set at r round

```
1:  $\mathcal{B} \leftarrow \{\}$ 
2: for  $p=0$  to 15,  $q=0$  to 31 do
3:    $\epsilon_d \leftarrow \{\}$ 
4:   for  $k_{idx} = 0$  to  $NumKey$  do
5:      $k \xleftarrow{\$} \{0,1\}^{256}$  ▷ Taking random key
6:      $ctr = 0$ 
7:     for  $iv_{idx} = 0$  to  $NumIV$  do
8:        $iv \xleftarrow{\$} \{0,1\}^{128}$ 
9:        $X \leftarrow$  Initialize with  $k, iv$ 
10:       $X' \leftarrow$  Initialize with  $k, iv \oplus \Delta_{i,j}^{(0)}$ 
11:       $Z, Z' \leftarrow Enc^{(r)}(X), Enc^{(r)}(X')$  ▷  $Enc^{(r)} :=$  Encryption of  $r$  rounds
12:       $\Delta_{p,q}^{(r)} \leftarrow Z \oplus Z'$  at  $X_{p,q}^{(r)}$ 
13:      if  $\Delta_{p,q}^{(r)} = 1$  then
14:         $ctr \leftarrow ctr + 1$ 
15:       $Pr \leftarrow ctr / NumIV$  ▷ calculating the probability
16:       $\epsilon_d \leftarrow \epsilon_d \cup \{2 \times (Pr - \frac{1}{2})\}$ 
17:       $\epsilon_d^* = \text{median of } \epsilon_d$ 
18:      if  $|\epsilon_d^*| > \epsilon'_d$  then
19:         $\mathcal{B} \leftarrow \mathcal{B} \cup \{\Delta_{p,q}^{(r)}\}$ 
20: return  $\mathcal{B}$ 
```

$\mathcal{ID} - \mathcal{OD}_l$. So, if the individual PNB sets P_l 's are already pre-computed, then one may use the above result to construct the PNBS.

The experimental result are given in Table 5 and Table 6 for ChaCha and Salsa respectively. Here we have taken the threshold $\epsilon'_d = 0.005$ and $\gamma = 0.07$ for both the ciphers. The experiments for PNBS are performed by taking $NumIV = 2^{34}$ times random initial vectors. Here we have observed that, the best case appears with $|\mathcal{B}| = 47$ and $|\mathcal{B}| = 25$, after applying Algorithm 1 and giving the initial input difference at $\Delta_{13,13}^{(0)}$ and $\Delta_{7,31}^{(0)}$ for ChaCha and Salsa respectively. These $\mathcal{ID}$ s are also considered in most of the previous works, so we take these specific $\mathcal{ID}$ s in all our experiments. This results 218 and 222 key bits after applying Algorithm 2, for the union of output bit differences, i.e., $\cup_{l=1}^{|\mathcal{B}|} \mathcal{OD}_l$, which are generated from the same $\mathcal{ID}$ for ChaCha and Salsa respectively.

4 Our Key Recovery Method

In the previous attack procedures [2], [24], [23], [18] the whole keyspace is divided into two parts (shown in Figure 3). The key recovery attack in this setup has been discussed in Section 2.4. We perform divide and conquer method by partitioning the keyspace into multiple smaller parts by means of multiple $\mathcal{OD}$'s with respect to a single $\mathcal{ID}$ (shown in Figure 4).

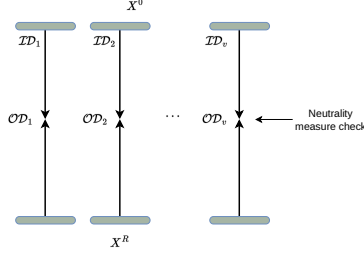


Fig. 1. Previous Construction Procedures of PNB sets

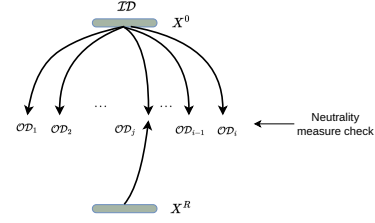


Fig. 2. Our Construction Procedure of PNB sets

Algorithm 2 FINDING_PNBS

Input: $\Delta_{i,j}^{(0)}$: $\mathcal{ID}$ at initial round; $\mathcal{B} = \{\mathcal{OD}_1, \dots, \mathcal{OD}_v\}$: Output difference set at r -th round; γ : threshold of the Neutrality measurement

Output: P : PNBS with respect to $\mathcal{B}$

- 1: $P \leftarrow \{\}$
- 2: **for** $k = 0$ to 256 **do**: ▷ Each key bit
- 3: $\gamma_{k_t} = 0$, for $t \in \mathcal{B}$
- 4: $key \xleftarrow{\$} \{0, 1\}^{256}$
- 5: $ctr_t = 0$, for $t \in \mathcal{B}$
- 6: **for** $iv_{idx} = 0$ to $NumIV$ **do**:
- 7: $iv \xleftarrow{\$} \{0, 1\}^{128}$
- 8: $iv' \leftarrow iv \oplus \Delta_{i,j}^{(0)}$ ▷ Giving initial input difference
- 9: $X \leftarrow$ Initialize state with key, iv
- 10: $X' \leftarrow$ Initialize state with key, iv'
- 11: $\Delta^{(r)} \leftarrow Enc^{(r)}(X) \oplus Enc^{(r)}(X')$ ▷ Storing the difference at r -th round
- 12: $Z, Z' \leftarrow X + Enc^{(R)}(X), X' + Enc^{(R)}(X')$
- 13: $key' \leftarrow$ Flip the k -th bit in key
- 14: $\bar{X} \leftarrow$ Initialize state with key', iv
- 15: $\bar{X}' \leftarrow$ Initialize state with key', iv'
- 16: $Y \leftarrow Dec^{(R-r)}(Z - \bar{X})$ using key', iv ▷ $Dec^{(R-r)} :=$ Decryption of $R - r$ rounds
- 17: $Y' \leftarrow Dec^{(R-r)}(Z' - \bar{X}')$ using key', iv'
- 18: $\Gamma \leftarrow Y \oplus Y'$
- 19: **for** $t = 1$ to v **do**:
- 20: **if** $((\Delta^{(r)} \oplus \Gamma) \text{ at } \mathcal{OD}_t)$ **then**:
- 21: $ctr_t \leftarrow ctr_t + 1$
- 22: **for** $t = 1$ to v **do**:
- 23: $Pr \leftarrow ctr_t / NumIV$ ▷ Calculating the probability
- 24: $\gamma_{k_t} \leftarrow 2 \times (Pr - \frac{1}{2})$
- 25: **if** $\gamma_{k_t} > \gamma$ **then**:
- 26: $P \leftarrow P \cup \{k\}$
- 27: **break**
- 28: **return** P

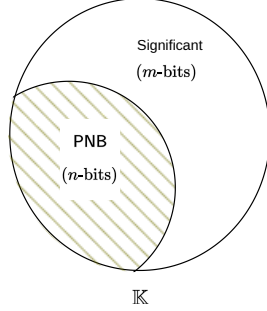


Fig. 3. Previous Key Recovery Attack

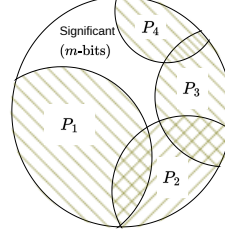


Fig. 4. New Key Recovery attack
 $P_t :=$ PNB set corresponding to $\mathcal{OD}_t$
 $m :=$ significant bits with respect to $\cup_{t=1}^4 P_t$

4.1 Partitioning the Keyspace

We devise our new multi-step key recovery attack procedure based on PNBS with respect to $\mathcal{B}' \subset \mathcal{B}$ with cardinality $|\mathcal{B}'| = v'$ and we label the $\mathcal{OD} \in \mathcal{B}'$ as $\mathcal{OD}_t$, $1 \leq t \leq v'$. This helps in dividing the keyspace into $v' + 1$ parts, of which v' parts correspond to the PNB bits corresponding to the v' $\mathcal{OD}$'s and one part is the significant key bits. This, in turn, reduces the total key space into smaller sub-key spaces that can be efficiently searched in a step by step manner. The selection criteria for proper choice of $\mathcal{B}'$ is discussed below and v' is discussed in Section 5.

Choice of the set $\mathcal{B}'$. For a fixed v' , the time complexity of the attack depends upon the cardinality of the PNB sets P_t , $t \in \{1, \dots, v'\}$ with respect to $\mathcal{OD}_t = \mathcal{OD}_l \in \mathcal{B}$, for some $l \in \{1, \dots, v\}$ and S , the significant bits corresponding to $\cup_{t=1}^{v'} P_t$. The time complexity becomes optimal if each PNB set P_t contains $256/(v' + 1)$ bits. In order to avoid redundancy in search, we need to divide the keyspace into mutually exclusive partitions. The selection process of the set $\mathcal{B}'$ and the corresponding mutually exclusive versions P'_t of the PNB sets P_t , is given in Algorithm 3.

4.2 Estimating Backward Bias

In the key recovery part of a PNB attack, ϵ_u plays the crucial role. We cannot derive ϵ_u directly as it requires large amount of samples to calculate the bias. Hence we approximate ϵ_u by using $\epsilon_d \cdot \epsilon_a$ for a single $\mathcal{ID} - \mathcal{OD}$. Here the product of the forward (ϵ_d) and backward (ϵ_a) bias events can be done, as the independent condition holds. In our attack approach, we have considered the union of PNB sets PNBS, for multiple $\mathcal{OD}$ bits in an intermediate round. The forward bias of each individual $\mathcal{OD}$ bits can be computed using the process used for single event. But at the time of calculating the backward bias, the question appears

Algorithm 3 Selection of $\mathcal{B}'$

Input: $\{P_l : 1 \leq l \leq |\mathcal{B}|\}$: P_l is PNB set corresponding to $\mathcal{OD}_l \in \mathcal{B}$; v' : A given number

Output: $\mathcal{B}'$ and the corresponding mutually exclusive sets P'_t 's.

- 1: $\mathcal{B}' \leftarrow \{\}$
 - 2: **for** $t = 1$ to v' **do**:
 - 3: Set $P'_t = P_l$ such that $|P_l|$ is nearest to $256/(v' + 1)$.
 - 4: $\mathcal{B}' \leftarrow \mathcal{B}' \cup \{\mathcal{OD}_l\}$
 - 5: $P_l = P_l \setminus P'_t, \forall l \leq |\mathcal{B}|$
 - 6: **return** $\mathcal{B}'$ and the corresponding set of mutually exclusive P'_t .
-

that, in which key bits one should give the random values so that the independent assumption still holds for the union event. We have found that, here the independent condition will hold, if the backward bias event for each $\mathcal{ID} - \mathcal{OD}_t$, is calculated after putting the random values in the key bits of PNBs i.e. taking the underlying set as $\cup_t P'_t$. We have validated in ToyChaCha that, using the above procedure, here one can approximate the practical value of ϵ_u for multiple of $\mathcal{OD}$ bits. The theoretical and practical values of ϵ_u corresponding to union of 2 and 3 $\mathcal{OD}$ s of ToyChaCha, are given in Table 2 and Table 4 respectively. Due to this, from a list of $\mathcal{ID} - \mathcal{OD}$ with $\epsilon_{d_t}, \epsilon_{a_t}$, one can not use our attack model, as to hold the independence assumption of the union event, one has to give the random values in the union of PNB sets. After putting the random values at PNBs and original values in the other key bits, we use Eq. 15 to compute ϵ_a for each $NumKey$ values. Here $NumIv = 2^{35}$ and $NumKey = 2^{10}$ is used to calculate ϵ_a corresponding to each $\mathcal{ID} - \mathcal{OD}_t$.

4.3 New Distinguisher Event and Data Complexity

In all earlier differential cryptanalysis of ChaCha and Salsa, the key recovery is based on the distinguisher event $\Gamma_{p,q} = 1$ corresponding to a single $\mathcal{OD}$ condition to $\Delta_{i,j}^{(0)} = 1$ (given in Eq. 16). In our approach, we consider union of v' many such events, denoted by $\Gamma_t = 1, 1 \leq t \leq v'$, corresponding to v' number of $\mathcal{OD}$ s in $\mathcal{B}'$.

Let $\Pr(\Gamma_t = 1 | \Delta_{i,j}^{(0)} = 1)$ be denoted by $\Pr_t = \frac{1}{2}(1 + \epsilon_t)$, where each bias ϵ_t can be approximated by (under independence assumptions as in the estimation of ϵ in Eq. (16)) the product of the corresponding individual forward and backward biases, say ϵ_{d_t} and ϵ_{a_t} respectively. These are the particular ϵ_d and ϵ_a values associated with the t -th $\mathcal{OD}$ (i.e. $\mathcal{OD}_t$) in $\mathcal{B}'$.

Our distinguisher event is that at least one of the Γ_t 's is 1 condition to $\Delta_{i,j}^{(0)} = 1$. The probability of our distinguisher $\Pr_u = \Pr\left(\bigvee_{t=1}^{v'} \Gamma_t = 1 | \Delta_{i,j}^{(0)} = 1\right)$ can be written as:

$$\Pr\left(\bigvee_{t=1}^{v'} \Gamma_t = 1 | \Delta_{i,j}^{(0)} = 1\right) = p_{rand}(1 + \epsilon_u), \quad (22)$$

where $p_{rand} = 1 - \frac{1}{2^{v'}}$ is the probability that the union of v' many uniformly distributed variables take the value 1. Note that $\Pr_u$ can also be approximated as $1 - \prod_{t=1}^{v'} (1 - \Pr_t) = 1 - \frac{1}{2^{v'}} \prod_{t=1}^{v'} (1 - \epsilon_t)$ under reasonable independence assumptions with respect to the underlying set is $\cup_{t=1}^{v'} P_t$, and hence the bias ϵ_u can be estimated as a function of the ϵ_{d_t} and ϵ_{a_t} values, $1 \leq t \leq v'$, as follows.

$$\epsilon_u = \frac{1 - \prod_{t=1}^{v'} (1 - \epsilon_{d_t} \cdot \epsilon_{a_t})}{2^{v'} - 1}. \quad (23)$$

Such independence assumptions are common in many attack strategies (such as DFA [28]) in the literature. For Salsa and ChaCha, such assumptions cannot be validated experimentally because of large sample and time complexity. One way of indirect validation could be via experimentation with a toy cipher, which we have performed. This is given in Section 6. To calculate ϵ_{a_t} corresponding to each t , we put random values in $\cup_{t=1}^{v'} P_t$ and check $\Pr(\Delta_{p_t, q_t}^{(r)} = \Gamma_{p_t, q_t} | \Delta_{i, j}^{(0)} = 1)$ in Eq. 15. Considering the same P_{fa} and P_{nd} as in Section 2.4, the number of samples required in our case can be written as

$$N_u \approx \left(\frac{\sqrt{\left(\frac{1-p_{rand}}{p_{rand}}\right) \alpha \log 4} + 3\sqrt{\frac{(1+\epsilon_u)(1-p_{rand}(1+\epsilon_u))}{p_{rand}}}}{\epsilon_u} \right)^2. \quad (24)$$

4.4 Modified Threshold for Key Recovery

Since we have proposed a new distinguisher event for key recovery, the threshold for the key votes need to be generalized accordingly. The analysis is similar to Section 2, except that Eq. (21) has to be modified with $\mu = N_u \Pr_u$ and $\sigma = \sqrt{N_u \Pr_u (1 - \Pr_u)}$. With necessary substitutions, the new threshold becomes

$$Th_u = N_u p_{rand} (1 + \epsilon_u) \pm 3\sqrt{N_u p_{rand} (1 + \epsilon_u) (1 - p_{rand} (1 + \epsilon_u))}. \quad (25)$$

4.5 Attack Procedure

Consider (Z, Z') are the keystream-faulty keystream pair for N_u samples with random IVs, and after giving the $\mathcal{ID}$ at $\Delta_{i, j}^{(0)}$. Suppose $Th_u(t)$ is the threshold for union of t PNB sets. Next, we discuss the steps of the key recovery process.

Step 1. Suppose m is the number of significant bits corresponding to the output difference set $\mathcal{B}'$. We put random values in $\cup_{t=1}^{v'} P'_t$ and all possible values in m bit positions and construct two initial states X and X' . After subtracting the constructed initial states from the generated keystream Z and the faulty keystream Z' , we decrypt for $R - r$ rounds and calculate the number of times the event $\bigvee_{t=1}^{v'} \Gamma_t = 1$ occurs. If the number satisfies the condition in Step 23 or Step 25 of Algorithm 4, then we

include the guessed key as a possible candidate key for the m significant bits. If the probability of false alarm is $2^{-\alpha}$, then the keyspace of m bits reduced from 2^m to $2^{m-\alpha}$.

Step 2. In this step, we consider the output differential set $\mathcal{B}'$ with $v' - 1$ number of $\mathcal{OD}$ s and the corresponding $\text{PNBS} \cup_{t=1}^{v'-1} P'_t$. Hence in this case, for each $2^{m-\alpha}$ possible keys, we put all possible values in $P'_{v'}$ bits and random values in other key bits. Now we calculate the number of times the event $\bigvee_{t=1}^{v'-1} \Gamma_t = 1$ occurs. If its greater than $Th_u(v' - 1)$ then we include it in the possible key set. Similarly as above, the search space reduces to $2^{m-\alpha+|P'_{v'}|-\alpha}$.

Step 3. Repeat the above Step 2 from the union of $v' - 2$ to 1.

Therefore using our attack strategy, the $\mathcal{B}'$ part of the keyspace is divided in to $v' + 1$ parts and we recover probable the key bits of each part by using the corresponding thresholds at each time. The key recovery attack procedure is discussed in Algorithm 4.

4.6 Time Complexity of the Proposed Attack

In Section 2.4, the time complexity of Eq. (19) is based on partitioning the 256-bit keyspace into two parts consists of m significant bits and $256 - m$ PNB bits. In our approach, we partition the keyspace into $v' + 1$ parts, of which one is the m many significant key bits, and the other v' parts are the mutually exclusive P'_t 's. Since for each of the 2^m possible assignment of the significant key bits, we test the distinguisher event in N samples, and then sequentially search the partitions P'_t 's exhaustively, the final complexity can be written as

$$T_u \approx 2^m \left(N + P_{fa} \left(\sum_{t=1}^{v'} 2^{|P'_t|} \right) \right). \quad (26)$$

However, as Eq. (19) is corrected in Eq. 20 by adding a term corresponding to the entire non-significant key-space, we can also refine T_u by adding v' more correction terms corresponding to the v' PNB partitions, as follows.

$$T_u \approx 2^m \left(N + P_{fa} \left(\sum_{t=1}^{v'} 2^{|P'_t|} \right) \right) + \sum_{t=1}^{v'} 2^{|P'_t|}. \quad (27)$$

Remark. It is worth noting that all our above formulations are generalization of (single $\mathcal{ID}$, single $\mathcal{OD}$)-based attack to (single $\mathcal{ID}$, multiple $\mathcal{OD}$) scenario. In particular, with $v' = 1$, Eq. (22), (23), (24), (26), (27) and (25) reduce to Eq. (16), (17), (18), (19), (20) and (21) respectively.

Algorithm 4 KEY_RECOVERY

Input: $\mathcal{B}' : \{\mathcal{OD}_t : t \leq v'\}$; $\mathcal{TH}$, $\mathcal{P}'$, $\mathcal{N}$: List of thresholds Th_u , mutually exclusive PNB sets P' (discussed in Algorithm 3), data complexities N respectively corresponding to $\cup_t \mathcal{OD}_t$ such that $t \leq v'$;
(Z, Z', iv): List of (Keystream, Faulty keystream, IV) tuples of size $\max\{\mathcal{N}\}$;
 $\Delta_{i,j}^{(0)}$: $\mathcal{ID}$ at initial round
Output: $\mathcal{K}$, set of possible keys

```
1:  $\mathcal{K} \leftarrow \{(0, \dots, 0)\}$  ▷ Initializing with zero key
2: for  $t = v'$  to 1 do:
3:    $P \leftarrow \cup_{l=1}^t \mathcal{P}'[l]$  ▷ PNBS corresponding to  $t$  unions
4:    $\mathcal{K}_{new} \leftarrow \{\}$ 
5:   if  $t == v'$  then
6:      $S \leftarrow \{0, \dots, 256\} \setminus P$  ▷ Significant key bits corresponding to  $v'$  unions
7:   else
8:      $S \leftarrow P'_{t+1}$ 
9:   for  $k \in \mathcal{K}$  do
10:    for  $k_{sig} = 0$  to  $2^{|S|} - 1$  do:
11:       $k_{tmp} \leftarrow \text{put } k_{sig} \text{ in } S \text{ bit positions}$ 
12:       $k_{tmp} \leftarrow k_{tmp} \oplus k$  ▷ generate a guessed key
13:       $k' \leftarrow \text{put random values in } P \text{ positions of } k_{tmp}$ 
14:       $ctr \leftarrow 0$ 
15:      for  $l = 0$  to  $\mathcal{N}[t]$  do:
16:         $iv' \leftarrow iv \oplus \Delta_{i,j}^{(0)}$  ▷ faulty  $IV$ 
17:         $Y \leftarrow \text{Dec}^{(R-r)}(Z[l], k', iv[l])$ 
18:         $Y' \leftarrow \text{Dec}^{(R-r)}(Z'[l], k', iv')$ 
19:        for  $m = 0$  to  $t$  do:
20:          if  $((Y \oplus Y') \text{ at } \mathcal{OD}_m)$  then:
21:             $ctr \leftarrow ctr + 1$ 
22:          break
23:          if  $((\mathcal{TH}[t] < \mathcal{N}[t] \cdot p_{rand}) \wedge (ctr < \mathcal{TH}[t]))$  then: ▷ For  $\epsilon_u < 0$  and
24:             $\mathcal{K}_{new} \leftarrow \mathcal{K}_{new} \cup \{k_{tmp}\}$ 
25:            if  $((\mathcal{TH}[t] > \mathcal{N}[t] \cdot p_{rand}) \wedge (ctr > \mathcal{TH}[t]))$  then: ▷ For  $\epsilon_u > 0$ 
26:               $\mathcal{K}_{new} \leftarrow \mathcal{K}_{new} \cup \{k_{tmp}\}$ 
27:    $\mathcal{K} \leftarrow \mathcal{K}_{new}$ 
28: return  $\mathcal{K}$ 
```

5 Our Attack Parameters and Results

All the previous works ([2], [23], [24], [15], [14], [21], [3], [15] etc.) have used the median ϵ_d^* of ϵ_d and median ϵ_a^* of ϵ_a to compute the complexities of their attacks. In [11], [26], [33], it is shown that, for fluctuating data, median is more “robust” than mean of some data samples. As discussed in Section 4.2, here we have to put random values in PNBS to calculate each backward bias in the union event. Now if the underlying set is $\cup_t P'_t$, then the backward bias (ϵ_a) of one $\mathcal{OD}$ would be affected due to the random values in the PNB sets of the

other $\mathcal{OD}$ s. In case of an alteration of sign in the calculated backward bias, the median of ϵ_a 's becomes very small. For example, for $\{-0.8, -0.75, -0.7, 0.01, 0.6, 0.65, 0.8\}$, of 7 biases, the median of the set becomes 0.01, which is very low comparative to the other high biases in the set. To mitigate this, we consider median of the absolute values to calculate the scenario. With this, the required number of samples becomes $\frac{1}{0.5 \times ((\epsilon_a)^*)^2}$, which leads to 2^{38} samples to calculate the backward bias of one $\mathcal{OD}$ with underlying set $\cup_t P'_t$ in our case. In all our experiments, we have considered 2^{45} samples (2^{35} random IVs for each of the 2^{10} random keys), which is more than sufficient to verify ϵ_a 's for the individual $\mathcal{ID} - \mathcal{OD}$. Now, due to the alteration of sign, instead of finding the median and using the values of ϵ_{dt} , ϵ_{at} in Eq. 23, we will calculate ϵ_u for each $NumKey$ values and take the median ϵ_u^* . This is used in Eqn. 24 to compute the final theoretical data complexity of the attack for the main and the toy cipher given in this paper. The individual biases at $\mathcal{ID} - \mathcal{OD}_i$'s are computed using 2^{35} random IVs and 2^{10} random keys. Here we have chosen $\alpha = 60$, so that after each step the key space reduces to 1. The experiments are performed in a HPC with 13 nodes, each having the specification Intel(R) Xeon(R) Gold 6148 CPU @ 2.40GHz with 40 CPUs.

5.1 Choice of v'

Here we use the terms $\epsilon_{u_{Pr}}$, $\epsilon_{u_{Th}}$ to address the practical (stated in Eq. 22) and theoretical (stated in Eq. 23) value of ϵ_u . The bias $\epsilon_{u_{Pr}}$ plays a crucial role to compute the data and the time complexities of the proposed attack. As $\epsilon_{u_{Th}}$ is an approximation relation under independence assumption, it is expected that it will deviate from the actual value $\epsilon_{u_{Pr}}$. The amount of deviation of error is likely to increase with the number of PNB sets used to construct new PNBS. This is also validated in ToyChaCha, given in Section 6. We have calculated the practical and theoretical values of ϵ_u , $\epsilon_{u_{Pr}}$ and $\epsilon_{u_{Th}}$, with respect to $v' = 2, 3$, and 4 in ToyChaCha. For a given $\epsilon_{u_{Th}}$, $\epsilon_{u_{Pr}}$, the percentage of relative approximation error for the PNBS (union of v' number of PNB sets) is calculated by:

$$\delta = \frac{|\epsilon_{u_{Th}} - \epsilon_{u_{Pr}}|}{\epsilon_{u_{Pr}}} \times 100.$$

We have found that, the value of δ is 3.4564%, 16.8571%, 72.8060% corresponding to union of 2, 3, 4 PNB sets respectively in the toy version. This verifies that the percentage of error δ increases with the union of the number of PNB sets. One should try to optimize v' , the number of union of PNB sets so that δ is less than 50%. At the same time, the time complexity will decrease if we partition the key space into more parts i.e. maximize v' . Therefore, we have to choose v' such that it optimized both δ and time complexity, as it becomes a joint optimization on v' . For ToyChaCha we have observed that, the error percentage is less than 50% up to $v'=3$. So, for full round cipher ChaCha and Salsa, we have given the results for $v' = 4$ in this paper. We have also computed the results corresponding to $v' = 3$ for all the ciphers, which are given in Appendix

C. We have given the results for ChaCha7, ChaCha7.5 in Section 5.2 and Salsa8 in Section 5.3. In [17], the authors first presented an attack on Salsa8.5, with the time complexity of $2^{245.84}$. Here, using our proposed method, we have computed the results for Salsa8.5 with respect to PNBS, but the time complexity of the key recovery attack becomes exhaustive for the cipher. However, for ChaCha7.5 and Salsa8.5, we have only used our key recovery procedure using PNBS, and not used any additional condition related to $\mathcal{HW}$ in the intermediate round or strategies which are applied in [34],[17]. Though our results on ChaCha7.5 beat the result of Flórez-Gutiérrez and Todo [20], the result reported in [34] remains better than both [20] and our current work. All the results of ChaCha7 and Salsa8 are given in Table 1.

5.2 ChaCha

We have discussed the result for 7 and 7.5 rounds using our key recovery attack procedure. ChaCha20/7 is exploited using differential approach and ChaCha7.5 is exploited using differential and differential-linear approaches, which we will see in the following. Here we have taken $\mathcal{ID} = \Delta_{13,13}^{(0)}$, as it is taken in most of the works to attack the cipher. We find the best result using our attack approach for $\epsilon_d = 0.005$ and $\gamma = 0.07$ for ChaCha7, ChaCha7.5 and Salsa8. We will discuss our results depending upon these parameters in the following.

7-round. We have observed output differential set $\mathcal{B}$ at $r = 3$ with $|\mathcal{B}| = 45$ for $\epsilon'_d = 0.005$. This gives $\mathcal{B}' = \{\Delta_{0,0}^{(3)}, \Delta_{3,0}^{(3)}, \Delta_{11,0}^{(3)}, \Delta_{2,0}^{(3)}\}$ after using Algorithm 3 and $v' = 4$. Here each P'_t is having cardinality $\{54, 40, 47, 16\}$ for $\mathcal{OD}_t \in \mathcal{B}'$. This results $\epsilon_u^* = 1.3331 \times 10^{-10}$. This gives the data and time complexities $2^{68.9008}$ and $2^{167.9008}$ respectively for 7 round of the cipher.

7.5-round. We have applied our attack procedure on 7.5 rounds of ChaCha cipher using differential and differential-linear approaches, which we discuss below. Here we have used approximately 2^{45} (specifically, $2^{44.322}$ samples with $NumIV = 2^{35}$ and $NumKey = 2^{9.322}$) for each approach to calculate the backward bias corresponding to each $\mathcal{OD}$.

Differential Approach. For PNBS consisting of union of 4, the forward bias ϵ_d is calculated after $r = 3$ round, and the backward bias ϵ_a is calculated after applying inverse round function for 4.5 round from $R = 7.5$ round. We get the best result for $\epsilon'_d = 0.005$ and $\gamma = 0.02$. The forward differential set $\mathcal{B}$ is having cardinality 45 for $\epsilon'_d = 0.005$. This gives $\mathcal{B}' = \{\Delta_{11,0}^{(3)}, \Delta_{0,0}^{(3)}, \Delta_{3,0}^{(3)}, \Delta_{2,0}^{(3)}\}$ corresponding to $v' = 4$. We partition the keyspace depending upon this $\mathcal{B}'$ and get $|P'_t| = \{38, 13, 11, 10\}$ (as discussed in Subsection 4.1). Here the final time and data complexity becomes $2^{53.4147}$, $2^{237.4147}$ respectively using our key recovery attack procedure.

Differential-linear Approach. In this case, to get the forward bias at $r = 3.5$, we use the differential-linear approach proposed in [7]. Consider the case $\epsilon_L = 1$ here. For this case, in our attack procedure, we have used $\mathcal{ID} = \Delta_{13,13}^{(0)}$ and found the $\mathcal{OD}$ bits, $\mathcal{B} = \{\Delta_{0,0}^{(3)}, \Delta_{2,0}^{(3)}, \Delta_{3,0}^{(3)}, \Delta_{11,0}^{(3)}, \Delta_{12,0}^{(3)}, \Delta_{15,0}^{(3)}\}$ corresponding to $\epsilon'_d > 0.005$, which are of the form $\Delta_{p,0}^{(3)}$ for some p -th word. These bits can be expressed as:

$$\left. \begin{aligned} \Delta_{0,0}^{(3)} &= \Delta_{0,0}^{(3.5)} \oplus \Delta_{5,12}^{(3.5)} \oplus \Delta_{10,0}^{(3.5)}, \\ \Delta_{2,0}^{(3)} &= \Delta_{2,0}^{(3.5)} \oplus \Delta_{7,12}^{(3.5)} \oplus \Delta_{8,0}^{(3.5)}, \\ \Delta_{3,0}^{(3)} &= \Delta_{3,0}^{(3.5)} \oplus \Delta_{4,12}^{(3.5)} \oplus \Delta_{9,0}^{(3.5)}, \\ \Delta_{11,0}^{(3)} &= \Delta_{11,0}^{(3.5)} \oplus \Delta_{12,0}^{(3.5)}, \\ \Delta_{12,0}^{(3)} &= \Delta_{12,16}^{(3.5)} \oplus \Delta_{1,0}^{(3.5)}, \\ \Delta_{15,0}^{(3)} &= \Delta_{15,16}^{(3.5)} \oplus \Delta_{0,0}^{(3.5)}, \end{aligned} \right\} \quad (28)$$

$$\left. \begin{aligned} \Delta_{0,0}^{(3)} &= \Delta_{0,0}^{(4)} \oplus \Delta_{5,7}^{(4)} \oplus \Delta_{5,19}^{(4)} \oplus \Delta_{10,12}^{(4)} \oplus \Delta_{15,0}^{(4)}, \\ \Delta_{2,0}^{(3)} &= \Delta_{2,0}^{(4)} \oplus \Delta_{7,7}^{(4)} \oplus \Delta_{7,19}^{(4)} \oplus \Delta_{8,12}^{(4)} \oplus \Delta_{13,0}^{(4)}, \\ \Delta_{3,0}^{(3)} &= \Delta_{3,0}^{(4)} \oplus \Delta_{4,7}^{(4)} \oplus \Delta_{4,19}^{(4)} \oplus \Delta_{9,12}^{(4)} \oplus \Delta_{14,0}^{(4)}, \\ \Delta_{11,0}^{(3)} &= \Delta_{12,0}^{(4)} \oplus \Delta_{11,0}^{(4)} \oplus \Delta_{12,8}^{(4)} \oplus \Delta_{1,0}^{(4)}, \\ \Delta_{12,0}^{(3)} &= \Delta_{12,24}^{(4)} \oplus \Delta_{1,16}^{(4)} \oplus \Delta_{1,0}^{(4)} \oplus \Delta_{11,0}^{(4)} \oplus \Delta_{6,7}^{(4)}, \\ \Delta_{15,0}^{(3)} &= \Delta_{15,24}^{(4)} \oplus \Delta_{0,16}^{(4)} \oplus \Delta_{0,0}^{(4)} \oplus \Delta_{10,0}^{(4)} \oplus \Delta_{5,7}^{(4)}. \end{aligned} \right\} \quad (29)$$

The Eqns. 28 are the linear combinations of the $\mathcal{OD}$ bits at 3.5 round with respect to the $\mathcal{OD}$ bits from $r = 3$ round. We have noticed that, these equations generate the same union of the PNB sets as it was generated for three rounds. As we get the best results in these specific intermediate bits only for the differential approach so we will get the best results for this case. Hence, the data and the time complexity will be same with the data and the time complexity of the differential approach for ChaCha7.5, which is $2^{53.4147}$ and $2^{237.4147}$ respectively. Therefore, using this approach, the differential attack can be viewed as a differential-linear attack, where one can improve the intermediate round, but the time complexity remains same.

We have also used the Eqns. 29 to compute forward bias at $r = 4$ round corresponding to the $\mathcal{OD}$ bits from $r = 3$ round. As discussed in the previous part, here also we will get the same $\mathcal{OD}$ bits and the PNBS also becomes same. Hence, the complexities will also remain the same with the above discussed part.

5.3 Salsa.

Here we have chosen $\mathcal{ID} = \Delta_{13,13}^{(0)}$ in the initial round and the proposed approach in section 3 is used to perform the key recovery attack on 8 rounds of the cipher. The output differential set $\mathcal{B}$ is formed at the intermediate round $r = 4$. This is used to find the PNB sets after the reduced final round $R = 8$. Here also we find the best time complexity for $\epsilon'_d = 0.005$ and $\gamma = 0.07$. This gives $|\mathcal{B}| = 25$ for $\epsilon'_d = 0.005$, and leads to $\mathcal{B}' = \{\Delta_{1,23}^{(4)}, \Delta_{1,4}^{(4)}, \Delta_{1,14}^{(4)}, \Delta_{6,15}^{(4)}\}$ for union of $v' = 4$ PNB sets. Here the mutually exclusive sets P'_t are having cardinality $\{42, 36, 30, 26\}$ corresponding to $\mathcal{OD}_t \in \mathcal{B}'$. This provides the final ϵ_u^* , data, time complexities equals -1.7114×10^{-9} , $2^{61.5361}$ and $2^{183.5361}$ respectively for $v' = 4$.

6 Practical Verification on ToyChaCha

Since the first PNB attack proposed by Aumasson et al. [2], there have been many works that have used approaches like differential, linear, differential-linear etc.

Using the updated strategies, the time complexities of the attack got reduced by approximately 2^{50} and 2^{40} times for ChaCha and Salsa respectively over almost two decades. Though the practical validation of the attack procedures still can not be done on either cipher.

In [14], the authors have introduced a toy version of ChaCha (ToyChaCha) and practically confirmed the attacks proposed by [2], [23] and finally validated the strategy proposed in their own work. Here we have used the same toy cipher to confirm our general method as well. In this section, we present practical implementation and verification of our attack on the ToyChaCha. The comparison table of our attack and others are shown in Table 3.

6.1 Structure of ToyChaCha

The structure of the toy cipher is similar to ChaCha128. Here, each word consists of 8 bits, which makes the total state size equal to 128. The key contains 4 words each of 8 bits, which is copied in the next row, following the same pattern of ChaCha128. The four constants of the initial round are chosen as $c_0 = 0\text{x}65$, $c_1 = 0\text{x}6\text{e}$, $c_2 = 0\text{x}32$, $c_3 = 0\text{x}74$. The quarter round of the round function takes 4 words (a, b, c, d) as input and outputs (a', b', c', d') , as follows.

$$\begin{aligned} a' &= a \boxplus b, & d' &= ((d \oplus a') \lll 4), \\ c' &= c \boxplus d', & b' &= ((b \oplus c') \lll 3), \\ a'' &= a' \boxplus b', & d'' &= ((d' \oplus a'') \lll 2), \\ c'' &= c' \boxplus d'', & b'' &= ((b' \oplus c'') \lll 1). \end{aligned} \tag{30}$$

Note that, here $\boxplus$ and $\lll$ are applied within 8 bit chunks.

6.2 Implementation of Our Attack

Here we attack 3.5 rounds of the ToyChaCha within which we find forward bias for 2 rounds and backward bias for 1.5 rounds. We have taken $\mathcal{ID} = \Delta_{13,0}^{(0)}$, $\epsilon'_d = 0.1$ and got the output differential set $\mathcal{B}$ with 54 bits in the intermediate round. Here $\mathcal{B}' = \{\Delta_{1,5}^{(2)}, \Delta_{2,0}^{(2)}\}$ is chosen with $\epsilon_d^* = -0.8619$ for $\mathcal{OD}_1 = \Delta_{1,5}^{(2)}$ and 0.7392 for $\mathcal{OD}_2 = \Delta_{2,0}^{(2)}$. The PNB sets P_i 's contains the key bits corresponding to the $\mathcal{OD}_i$ s with $i \in \{1, 2\}$, which are $\{0, 1, 2, 3, 4, 5, 6, 7, 29, 30, 31\}$ and $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 23, 28, 29, 30, 31\}$ respectively with respect to $\gamma = 0.7$. This gives $|P_1 \cup P_2| = 21$. The backward biases ϵ_a^* corresponding to $\mathcal{OD}_1$ and $\mathcal{B}'$ are 0.002563, 1.0 respectively. Both the theoretical and practical values of ϵ_u^* becomes 0.2462. Hence we got the sample space $N_u = 2^{8.517}$ (by applying Eq. (24)) for the union of two sets for $\alpha = 30$. We have calculated the thresholds (defined in Section 4.4) depending upon the theoretical ϵ_u^* and obtained $Th_u(2) = 328$ and $Th_u(1) = 12$ corresponding to $P_1 \cup P_2$ and P_1 respectively. Now, the significant bits with respect to $P_1 \cup P_2$ contains 11 bits. After using the threshold $Th_u(2)$ in the key recovery attack, the keyspace is reduced to 1 from 2^{11} . In the

next step, we use the threshold $Th_u(1)$ with respect to P_1 . Now, the key space corresponding to P_1 becomes $2^{10} \times 1$. Using the threshold $Th_u(1)$, the significant key space with respect to P_1 reduced to 2^1 from 2^{10} . Hence, the final practical time complexity using our attack procedure becomes, $T_{u_{Pr}} = 2^{19.6824}$. In the case of theoretical time complexity ($T_{u_{Th}}$), the key space reduces to 1 after using each threshold (using $\alpha = 30$). Hence the theoretical time complexity becomes, $T_{u_{Th}} = 2^{19.6789}$. To calculate all the biases in the ToyChaCha, we have used $NumIV = 2^{22}$ and $NumKey = 2^{10}$. Here we have experimentally observed that, the original key remains in 96% of cases in the final reduced key space. The details of the key recovery attack in some other $\mathcal{ID} - \mathcal{OD}$ bits are enlisted in Table 2. The authors in [14] showed key space reduced to $2^{24.65}$ using Aumasson's work and the time complexity using their own attack reduced to $2^{16.15}$. Here the original key remains in 99.81% and 99.71% cases for [2] and [14] respectively. The comparison of our work with all the other works in ToyChaCha is given in Table 3.

$B' = \{\mathcal{OD}_i, \mathcal{OD}_j\}$	$ P_i \cup P_j $	ϵ_d^* for $\{\mathcal{OD}_i, \mathcal{OD}_j\}$	ϵ_a^* for $\{\mathcal{OD}_i, \mathcal{OD}_j\}$	$\epsilon_{u_{Th}}^*$	$\epsilon_{u_{Pr}}^*$	$N_{u_{Th}}$	$N_{u_{Pr}}$	$T_{u_{Th}}$	$T_{u_{Pr}}$	Success rate (%)
$\{\Delta_{1,5}, \Delta_{2,0}\}$	21	{-0.8619, 0.7392}	{0.002563, 1.0}	0.2462	0.2462	$2^{8.517}$	$2^{8.517}$	$2^{19.6789}$	$2^{19.6824}$	96
$\{\Delta_{1,6}, \Delta_{2,0}\}$	21	{0.9159, 0.7392}	{0.004039, 1.0}	0.2467	0.2467	$2^{8.509}$	$2^{8.509}$	$2^{19.7609}$	$2^{19.7609}$	97
$\{\Delta_{1,6}, \Delta_{3,4}\}$	22	{0.9159, 0.6378}	{0.006117, 0.006418}	0.003195	0.00377	$2^{21.4722}$	$2^{20.994}$	$2^{31.4722}$	$2^{31.446}$	90
$\{\Delta_{2,7}, \Delta_{2,0}\}$	21	{-0.2577, 0.7392}	{0.03725, 1.0}	0.2456	0.2461	$2^{10.361}$	$2^{10.369}$	$2^{22.55}$	$2^{22.55}$	99
$\{\Delta_{14,5}, \Delta_{2,0}\}$	21	{-0.2841, 0.7392}	{0.007886, 1.0}	0.2462	0.2459	$2^{10.073}$	$2^{10.06}$	$2^{20.823}$	$2^{20.823}$	99

Table 2. Experimental results corresponding to union of 2 $\mathcal{OD}$ s.

Here we have applied Eqn. 23 with $v' = 3$ also for ToyChaCha and enlisted some of the triplets in Table 4. All the biases in this table also are calculated corresponding to $NumIV = 2^{24}$ and $NumKey = 2^{10}$.

$\mathcal{ID}$	$\mathcal{OD}$	ϵ_d^*	ϵ_a^*	$\epsilon_{u_{Th}}^*$	$\epsilon_{u_{Pr}}^*$	N_u	Success Rate (%)	$T_{u_{Pr}}$	$T_{u_{Th}}$	Refs.
$\Delta_{13,0}$	$\{\Delta_{1,5}, \Delta_{2,0}\}^\S$	{-0.8619, 0.7392}	{0.002563, 1.0}	0.2462	0.2462	$2^{8.517}$	96	$2^{19.6824}$	$2^{19.6789}$	This work
$\Delta_{13,0}$	$\Delta_{1,6}$	0.9167	0.377	0.343	0.3652	$2^{8.562}$	99.81	$2^{23.60}$	$2^{24.67}$	
$\Delta_{13,0}$	$\Delta_{1,6}$	0.98	0.49	0.4802	0.3814	$2^{7.531}$	99.71	$2^{23.50}$	$2^{24.65}$	[14]
$\{\Delta_{13,0}, \Delta_{14,0}, \Delta_{15,0}\}$	$\{\Delta_{1,6}, \Delta_{2,6}, \Delta_{3,6}\}^\dagger$	0.91	1	0.91	-	$2^{6.56}$	-	$2^{15.1}$	$2^{16.15}$	

§ Involves $\mathcal{ID} - \mathcal{OD}_t$, for $t \in \{0, 1\}$ pairs. ϵ_d^* , ϵ_a^* at t -th time is given in the t -th position.

† Involves $\mathcal{ID}_t - \mathcal{OD}_t$, for $t \in \{0, 1, 2\}$ pairs.

Table 3. Summary of key recovery attack procedures applied on ToyChaCha.

7 Conclusion

This work provides a new approach of constructing the PNB set in terms of the union of the output bit differentials ($\mathcal{OD}$) for the same input bit differential ($\mathcal{ID}$) at the initial round of the ciphers. This increases the cardinality of the

$B' = \{\mathcal{OD}_i, \mathcal{OD}_j, \mathcal{OD}_k\}$	ϵ_d^* for $\{\mathcal{OD}_i, \mathcal{OD}_j, \mathcal{OD}_k\}$	ϵ_a^* for $\{\mathcal{OD}_i, \mathcal{OD}_j, \mathcal{OD}_k\}$	$\epsilon_{u_{Th}}^*$	$\epsilon_{u_{Pr}}^*$
$\Delta_{1,3}, \Delta_{12,5}, \Delta_{14,6}$	-0.5818, 0.5872, 0.6378	0.0008023, 0.002579, 0.0009301	0.0002137	0.000228
$\Delta_{1,4}, \Delta_{2,5}, \Delta_{12,5}$	-0.7619, 0.7142, 0.5872	0.0003426, 0.001488, 0.002562	0.0003143	0.0003506
$\Delta_{1,4}, \Delta_{12,5}, \Delta_{14,6}$	-0.7619, 0.5872, 0.6378	0.0003746, 0.002566, 0.0008495	0.0002513	0.0002473
$\Delta_{1,5}, \Delta_{2,5}, \Delta_{12,5}$	-0.8618, 0.7142, 0.5872	0.0004065, 0.001530, 0.002629	0.0003153	0.000316
$\Delta_{1,5}, \Delta_{12,5}, \Delta_{14,6}$	-0.8619, 0.5872, 0.6378	0.0003510, 0.002568, 0.0008187	0.0002453	0.0002351
$\Delta_{1,6}, \Delta_{2,5}, \Delta_{12,5}$	0.9159, 0.7142, 0.5872	0.001040, 0.001539, 0.002642	0.0005036	0.0005272
$\Delta_{1,6}, \Delta_{3,4}, \Delta_{12,5}$	0.9159, 0.6378, 0.5872	0.001944, 0.0002799, 0.001167	0.000373	0.0003968

Table 4. $\epsilon_{u_{Th}}^*$ vs $\epsilon_{u_{Pr}}^*$ for union of 3 $\mathcal{OD}$ s.

PNB set. Further, the keyspace can be partitioned into smaller parts leading to more efficient key recovery.

One interesting related problem is: how many and which $\mathcal{OD}$'s should we choose to perform the attack most efficiently. This depends on the relative cardinalities of the PNBs corresponding to the $\mathcal{OD}$ s and hence poses an important optimization problem. In our work, we have taken $v' = 4$ and four specific $\mathcal{OD}$'s. Due to approximation error in independence assumptions, it is expected that the union of more PNBs we take, the average error percentage would also keep increasing. We have verified this fact using **ToyChaCha** as well. Hence we have to choose the number PNBs (which is equal to the number of $\mathcal{OD}$ s) to be united judiciously. We have shown that if we take 4 $\mathcal{OD}$ s for **ToyChaCha**, then the average error becomes much higher than that for 3 $\mathcal{OD}$ s. Hence, we have considered the union of 2 and 3 $\mathcal{OD}$ s for **ToyChaCha** and 4 $\mathcal{OD}$ s for original **ChaCha** and **Salsa** ciphers. To demonstrate our attack procedure on **ToyChaCha**, we have used 2 $\mathcal{OD}$ s as we get the best time complexity using $v' = 2$.

Our attack achieves a time complexity of $2^{167.9008}$ for **ChaCha20/7**, $2^{237.4147}$ for **ChaCha20/7.5** and $2^{183.5361}$ for **Salsa20/8**, significantly improving all the existing PNB-based attacks for 7 and 8 rounds on these ciphers. Further, the time complexity of **ChaCha7.5** using our proposed method is better than the time complexity of the latest published work by Flórez-Gutiérrez and Todo (Eurocrypt 2025), which is $2^{250.2}$. As future work, it would be interesting to combine the special techniques like $\mathcal{HW}$ in the intermediate round and other feasible conditions with our proposed method using PNBS and investigate whether it improves the time Complexity for the key recovery attack.

Exhaustively enumerating the attack complexities for the other variations of the ciphers and for different fixed parameters such as $\epsilon'_d, \gamma, \alpha$ can also be taken up as another important future work.

Acknowledgments

We are grateful to Shashi Prabhakar, Physical Research Laboratory, Ahmedabad for his valuable inputs on our computation-intensive experiments.

References

1. the ecrypt stream cipher project. estream portfolio of stream ciphers. <https://www.ecrypt.eu.org/stream/>
2. Aumasson, J.P., Fischer, S., Khazaei, S., Meier, W., Rechberger, C.: New features of latin dances: analysis of salsa, chacha, and rumba. In: Fast Software Encryption: 15th International Workshop, FSE 2008, Lausanne, Switzerland, February 10-13, 2008, Revised Selected Papers 15. pp. 470–488. Springer (2008)
3. Beierle, C., Broll, M., Canale, F., David, N., Flórez-Gutiérrez, A., Leander, G., Naya-Plasencia, M., Todo, Y.: Improved differential-linear attacks with applications to arx ciphers. *Journal of Cryptology* **35**(4), 29 (2022)
4. Bellini, E., Gérault, D., Grados, J., Makarim, R.H., Peyrin, T.: Boosting differential-linear cryptanalysis of chacha7 with MILP. *IACR Trans. Symmetric Cryptol.* **2023**(2), 189–223 (2023), <https://doi.org/10.46586/tosc.v2023.i2.189-223>
5. Bernstein, D.J.: The salsa20 family of stream ciphers. In: New stream cipher designs: the eSTREAM finalists, pp. 84–97. Springer (2008)
6. Bernstein, D.J., et al.: Chacha, a variant of salsa20. In: Workshop record of SASC. vol. 8, pp. 3–5. Citeseer (2008)
7. Choudhuri, A.R., Maitra, S.: Significantly improved multi-bit differentials for reduced round salsa and chacha. *IACR Transactions on Symmetric Cryptology* pp. 261–287 (2016)
8. Coutinho, M., Passos, I., Vásquez, J.C.G., Sarkar, S., de Mendonça, F.L., de Sousa Jr, R.T., Borges, F.: Latin dances reloaded: Improved cryptanalysis against salsa and chacha, and the proposal of forró. *Journal of Cryptology* **36**(3), 18 (2023)
9. Coutinho, M., Souza Neto, T.C.: Improved linear approximations to arx ciphers and attacks against chacha. In: Advances in Cryptology–EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part I 40. pp. 711–740. Springer (2021)
10. Crowley, P.: Truncated differential cryptanalysis of five rounds of salsa20. *Cryptology ePrint Archive* (2005)
11. DeGroot, M.H., Schervish, M.J.: Probability and Statistics. Addison Wesley, 3rd edn. (2002)
12. Dey, C., Dey, S., Girme, R., Sarkar, S.: Enhancing the key recovery attack on round reduced salsa. *IEEE Access* **12**, 31736–31744 (2024), <https://doi.org/10.1109/ACCESS.2024.3367797>
13. Dey, S.: Advancing the idea of probabilistic neutral bits: First key recovery attack on 7.5 round chacha. *IEEE Trans. Inf. Theory* **70**(8), 6091–6106 (2024), <https://doi.org/10.1109/TIT.2024.3389874>
14. Dey, S., Garai, H.K., Maitra, S.: Cryptanalysis of reduced round chacha - new attack & deeper analysis. *IACR Trans. Symmetric Cryptol.* **2023**(1), 89–110 (2023), <https://doi.org/10.46586/tosc.v2023.i1.89-110>
15. Dey, S., Garai, H.K., Sarkar, S., Sharma, N.K.: Revamped differential-linear cryptanalysis on reduced round chacha. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 86–114. Springer (2022)
16. Dey, S., Garai, H.K., Sarkar, S., Sharma, N.K.: Enhanced differential-linear attacks on reduced round chacha. *IEEE Trans. Inf. Theory* **69**(8), 5318–5336 (2023), <https://doi.org/10.1109/TIT.2023.3269790>

17. Dey, S., Maitra, S., Sarkar, S., Sharma, N.K.: Significantly improved cryptanalysis of salsa20 with two-round criteria. *IACR Trans. Symmetric Cryptol.* **2025**(1), 420–443 (2025)
18. Dey, S., Sarkar, S.: Improved analysis for reduced round salsa and chacha. *Discrete Applied Mathematics* **227**, 58–69 (2017)
19. Fischer, S., Meier, W., Berbain, C., Biasse, J.F., Robshaw, M.J.: Non-randomness in estream candidates salsa20 and tsc-4. In: *Progress in Cryptology-INDOCRYPT 2006: 7th International Conference on Cryptology in India, Kolkata, India, December 11-13, 2006. Proceedings 7*. pp. 2–16. Springer (2006)
20. Flórez-Gutiérrez, A., Todo, Y.: Improved cryptanalysis of chacha: Beating pnbs with bit puncturing. In: *EUROCRYPT (1). Lecture Notes in Computer Science*, vol. 15601, pp. 427–457. Springer (2025)
21. Garai, H.K., Dey, S.: A multi-step key recovery attack on reduced round salsa and chacha. *Cryptologia* pp. 1–16 (2024)
22. Ishiguro, T., Kiyomoto, S., Miyake, Y.: Latin dances revisited: new analytic results of salsa20 and chacha. In: *Information and Communications Security: 13th International Conference, ICICS 2011, Beijing, China, November 23-26, 2011. Proceedings 13*. pp. 255–266. Springer (2011)
23. Maitra, S.: Chosen iv cryptanalysis on reduced round chacha and salsa. *Discrete Applied Mathematics* **208**, 88–97 (2016)
24. Maitra, S., Paul, G., Meier, W.: Salsa20 cryptanalysis: new moves and revisiting old styles. *Cryptology ePrint Archive* (2015)
25. Miyashita, S., Ito, R., Miyaji, A.: Pnb-focused differential cryptanalysis of chacha stream cipher. In: *Australasian Conference on Information Security and Privacy*. pp. 46–66. Springer (2022)
26. Moore, D.S., McCabe, G.P., Craig, B.A.: *Introduction to the Practice of Statistics*. W. H. Freeman, New York, 10 edn. (2021)
27. Mouha, N., Preneel, B.: Towards finding optimal differential characteristics for arx: Application to salsa20. *Cryptology ePrint Archive* (2013)
28. Sakiyama, K., Sasaki, Y., Li, Y.: *Security of Block Ciphers - From Algorithm Design to Hardware Implementation*. Wiley (2015)
29. Sharma, N.K., Dey, S., Sarkar, S., Maitra, S.: On improved cryptanalytic results against chacha for reduced rounds ≥ 7 . In: Mukhopadhyay, S., Stanica, P. (eds.) *Progress in Cryptology - INDOCRYPT 2024 - 25th International Conference on Cryptology in India, Chennai, India, December 18-21, 2024, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 15496, pp. 29–52. Springer (2024), https://doi.org/10.1007/978-3-031-80311-6_2
30. Shi, Z., Zhang, B., Feng, D., Wu, W.: Improved key recovery attacks on reduced-round salsa20 and chacha. In: *International Conference on Information Security and Cryptology*. pp. 337–351. Springer (2012)
31. Tsunoo, Y., Saito, T., Kubo, H., Suzaki, T., Nakashima, H.: Differential cryptanalysis of salsa20/8. In: *Workshop Record of SASC*. vol. 28 (2007)
32. Wang, S., Liu, M., Hou, S., Lin, D.: Moving a step of chacha in syncopated rhythm. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. Lecture Notes in Computer Science*, vol. 14083, pp. 273–304. Springer (2023), https://doi.org/10.1007/978-3-031-38548-3_10
33. Witte, R., Witte, J.: *Statistics*. Wiley (2017), <https://books.google.co.in/books?id=KcxjDwAAQBAJ>

34. Xu, Z., Xu, H., Tan, L., Qi, W.: Differential-linear cryptanalysis of reduced round chacha. IACR Trans. Symmetric Cryptol. **2024**(2), 166–189 (2024)

Appendix

$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$	$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$	$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$	$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$
384	45	186	29	416	43	184	27	448	46	193	38	480	44	185	28
385	48	217	91	417	49	217	91	449	47	216	91	481	46	217	91
386	60	221	79	418	63	225	79	450	60	226	79	482	62	223	79
387	65	216	53	419	65	217	47	451	61	211	41	483	67	219	54
388	44	182	24	420	43	182	23	452	40	173	24	484	43	181	24
389	47	207	54	421	45	205	54	453	45	204	54	485	45	204	54
390	56	222	54	422	55	222	54	454	59	221	54	486	56	220	54
391	65	220	41	423	64	220	41	455	66	221	41	487	62	219	41
392	45	184	28	424	45	186	28	456	45	185	28	488	46	186	28
393	49	222	91	425	47	219	91	457	47	217	91	489	48	221	91
394	55	223	79	426	55	223	80	458	55	223	80	490	54	224	80
395	51	204	41	427	50	203	41	459	52	204	41	491	53	214	54
396	40	178	28	428	46	180	27	460	42	178	28	492	42	186	27
397	44	214	91	429	47	218	91	461	43	211	91	493	45	212	91
398	58	227	80	430	57	226	80	462	57	226	80	494	59	226	80
399	76	234	80	431	76	234	80	463	77	234	80	495	76	234	80
400	43	180	38	432	43	180	39	464	46	188	38	496	38	167	38
401	45	218	91	433	46	222	91	465	44	217	91	497	44	218	91
402	55	232	80	434	58	233	80	466	56	233	80	498	57	234	80
403	49	209	41	435	54	218	53	467	52	215	41	499	54	215	42
404	47	184	28	436	45	179	26	468	44	178	27	500	43	178	28
405	43	197	54	437	43	197	54	469	44	201	54	501	45	203	54
406	54	225	54	438	55	225	54	470	53	223	55	502	50	219	54
407	50	203	54	439	53	211	54	471	53	210	54	503	49	203	54
408	47	193	42	440	48	195	41	472	46	193	41	504	47	194	41
409	50	215	91	441	49	212	91	473	49	215	91	505	49	212	91
410	56	225	80	442	57	224	80	474	56	225	80	506	57	225	80
411	54	212	54	443	53	211	53	475	57	217	53	507	56	218	55
412	46	190	28	444	44	180	27	476	45	191	28	508	47	193	38
413	45	221	91	445	46	221	91	477	46	221	91	509	46	221	91
414	49	225	80	446	52	225	80	478	49	225	80	510	49	225	80
415	49	211	54	447	47	209	54	479	47	209	54	511	47	209	53

Table 5. Number of union of PNB bits of P_i for $\mathcal{OD}_i$ corresponding to all possible $\mathcal{ID}$ bits for ChaCha. The forward bias and the PNB bits are calculated corresponding to the thresholds $\epsilon'_d = 0.005$ and $\gamma = 0.07$ respectively.

A Union of PNB Sets for ChaCha

The finding PNB bits experiment is done with respect to the $NumIV = 2^{34}$ and the backward bias with respect to sample size $= 2^{35} \times 2^{10}$. Here the input difference is given at $\Delta_{13,13}^{(0)}$ in the initial state. The output differential set, $\mathcal{B}$ is generated with $|\mathcal{B}| = 26$. The ϵ'_d and γ parameters are taken as 0.005 and 0.07 as discussed above in the Section 5. The exclusive PNB sets for $\mathcal{B}' = \{\Delta_{0,0}^{(3)}, \Delta_{3,0}^{(3)}, \Delta_{11,0}^{(3)}, \Delta_{2,0}^{(3)}\}$ are given in the following:

$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$	$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$	$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$	$\mathcal{ID}$	$\#\mathcal{OD}$	$ \cup_{i \in \mathcal{OD}} P_i $	$\max(P_i)$
192	-	-	-	224	16	195	41	256	15	173	36	288	2	39	33
193	-	-	-	225	15	196	43	257	15	175	37	289	1	28	28
194	-	-	-	226	16	201	43	258	16	176	38	290	1	30	30
195	-	-	-	227	17	199	43	259	17	175	37	291	1	30	30
196	-	-	-	228	17	197	42	260	16	173	37	292	1	29	29
197	-	-	-	229	17	200	43	261	15	176	37	293	1	29	29
198	-	-	-	230	16	193	42	262	16	171	37	294	1	28	28
199	-	-	-	231	15	199	41	263	16	177	36	295	1	28	28
200	-	-	-	232	15	203	42	264	18	191	36	296	2	36	29
201	-	-	-	233	15	203	42	265	18	190	36	297	2	37	30
202	-	-	-	234	15	197	42	266	15	174	36	298	1	29	29
203	-	-	-	235	16	197	42	267	17	177	38	299	1	28	28
204	-	-	-	236	17	214	43	268	17	187	37	300	1	32	32
205	-	-	-	237	17	201	42	269	17	181	37	301	1	32	32
206	-	-	-	238	15	197	43	270	15	179	37	302	2	37	32
207	-	-	-	239	15	198	43	271	15	173	37	303	1	28	28
208	-	-	-	240	16	200	42	272	15	172	37	304	1	28	28
209	-	-	-	241	19	218	42	273	19	191	36	305	1	28	28
210	-	-	-	242	19	221	42	274	18	192	36	306	1	32	32
211	-	-	-	243	16	197	41	275	15	171	36	307	2	38	32
212	-	-	-	244	15	199	42	276	15	173	36	308	2	37	32
213	-	-	-	245	15	200	42	277	17	187	36	309	2	36	29
214	-	-	-	246	17	201	42	278	18	188	37	310	2	36	29
215	-	-	-	247	16	198	40	279	15	168	34	311	2	35	30
216	-	-	-	248	15	196	43	280	15	173	37	312	1	27	27
217	-	-	-	249	15	197	42	281	16	174	37	313	1	28	28
218	-	-	-	250	15	198	43	282	16	176	37	314	1	28	28
219	-	-	-	251	15	201	43	283	16	177	37	315	1	30	30
220	-	-	-	252	15	203	42	284	15	177	37	316	1	30	30
221	-	-	-	253	17	201	42	285	17	178	37	317	1	31	31
222	-	-	-	254	18	215	42	286	18	190	36	318	1	29	29
223	-	-	-	255	25	222	42	287	25	202	37	319	2	40	33

Table 6. Number of union of PNB bits of P_i for $\mathcal{OD}_i$ corresponding to all possible $\mathcal{ID}$ bits for **Salsa**. The forward bias and the PNB bits are calculated with respect to the thresholds $\epsilon'_d = 0.005$ and $\gamma = 0.07$ respectively.

$P'_1 = \{0, 1, 2, 3, 6, 7, 15, 16, 19, 22, 23, 27, 31, 32, 35, 38, 39, 40, 41, 42, 51, 52, 55, 56, 59, 60, 61, 62, 63, 66, 78, 90, 95, 103, 115, 123, 140, 152, 155, 156, 159, 160, 161, 162, 163, 180, 181, 182, 191, 199, 219, 223, 224, 244\}$, $|P'_1| = 54$,

$P'_2 = \{128, 129, 130, 131, 8, 9, 10, 20, 148, 149, 150, 24, 28, 29, 30, 34, 167, 46, 58, 187, 192, 71, 83, 212, 91, 96, 97, 98, 99, 102, 236, 127, 111, 112, 118, 119, 248, 251, 252, 255\}$, $|P'_2| = 40$,

$P'_3 = \{4, 136, 137, 138, 11, 139, 144, 17, 18, 157, 36, 47, 176, 188, 64, 193, 67, 68, 69, 72, 73, 74, 79, 213, 216, 92, 93, 94, 220, 225, 226, 227, 100, 101, 228, 104, 105, 106, 107, 232, 250, 240, 243, 249, 122, 253, 254\}$, $|P'_3| = 47$,

$P'_4 = \{65, 70, 135, 204, 14, 80, 86, 87, 26, 116, 245, 246, 120, 124, 125, 126\}$, $|P'_4| = 16$.

B Union of PNB Sets for Salsa

Here also the experiments for finding the PNB sets are done with respect to the $NumIV = 2^{34}$ and the backward bias corresponding to sample size $= 2^{35} \times 2^{10}$. Here the input difference is given at $\Delta_{7,31}^{(0)}$ in the initial state. The output differential set $\mathcal{B}$, with $|\mathcal{B}| = 17$. The ϵ'_d and γ parameters are taken as 0.005 and 0.07 as discussed above in the Section 5. The exclusive PNB sets for $\mathcal{B}' = \{\Delta_{1,23}^{(4)}, \Delta_{1,4}^{(4)}, \Delta_{1,14}^{(4)}, \Delta_{6,15}^{(4)}\}$ are given in the following:

$P'_1 = \{3, 4, 5, 6, 7, 80, 81, 82, 97, 98, 99, 100, 117, 157, 158, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 219, 220, 221, 222, 223, 224, 233, 234, 235, 251, 252, 253, 254, 255\}$, $|P'_1| = 42$,

$P'_2 = \{138, 139, 16, 17, 18, 19, 20, 160, 161, 162, 163, 164, 165, 166, 167, 187, 188, 189, 190, 191, 200, 201, 202, 203, 93, 94, 95, 232, 236, 237, 110, 111, 112, 113, 246, 247\}$, $|P'_2| = 36$,

$P'_3 = \{71, 72, 73, 210, 211, 148, 212, 213, 26, 27, 28, 29, 30, 31, 225, 226, 168, 169, 170, 171, 172, 173, 108, 242, 243, 244, 245, 120, 121, 122\}$, $|P'_3| = 30$,

$P'_4 = \{128, 129, 65, 66, 8, 9, 10, 11, 12, 13, 14, 15, 142, 143, 83, 84, 85, 86, 87, 88, 104, 105, 51, 52, 53, 54\}$, $|P'_4| = 26$.

C Results of PNBs consist in union of 3 PNB Sets

For ChaCha20/7. Here also the output differential set $\mathcal{B}$ remains same with the union of 4 PNB sets. For $v' = 3$, the reduced output differential set $\mathcal{B}'$ contains $\{\Delta_{0,0}^{(3)}, \Delta_{11,0}^{(3)}, \Delta_{3,0}^{(3)}\}$ for $\epsilon'_d = 0.005$ and $P'_t = \{54, 67, 20\}$ for $\mathcal{OD}_t \in \mathcal{B}'$ i.e. total 141 key bits remain in $\cup_t P'_t$. This gives the value of ϵ_u^* , final data, time complexities equals 2.1408×10^{-9} , $2^{61.9897}$ and $2^{176.9897}$ for ChaCha20/7.

For ChaCha20/7.5. For the union of 3 $\mathcal{OD}$ s, using the same procedure, we get $B' = \{\Delta_{11,0}^{(3)}, \Delta_{0,0}^{(3)}, \Delta_{2,0}^{(3)}\}$ for using the differential approach. This results the exclusive PNB sets $P'_t = \{38, 13, 11\}$ corresponding to union of 3 $\mathcal{OD}$ s. In this case, the final time complexities using our attack approach becomes exhaustive i.e. 2^{256} .

For the differential-linear approach, using the union of 3 $\mathcal{OD}$ s also, we get the $\mathcal{OD}$ bits and the PNB sets same with the differential approach. Hence for this case also (like $v' = 4$), the data and the time complexities will remain same.

For Salsa20/8. For $v' = 3$, we find $\mathcal{B}' = \{\Delta_{1,23}^{(4)}, \Delta_{1,4}^{(4)}, \Delta_{1,14}^{(4)}\}$. This results the final ϵ_u^* , data, time complexities equals 3.1321×10^{-9} , $2^{60.8918}$ and $2^{208.8918}$ for union of 3 PNB sets.